

Lesson 4: Concept and Category Rule Definitions



4.0 Lesson Overview



Lesson Overview

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



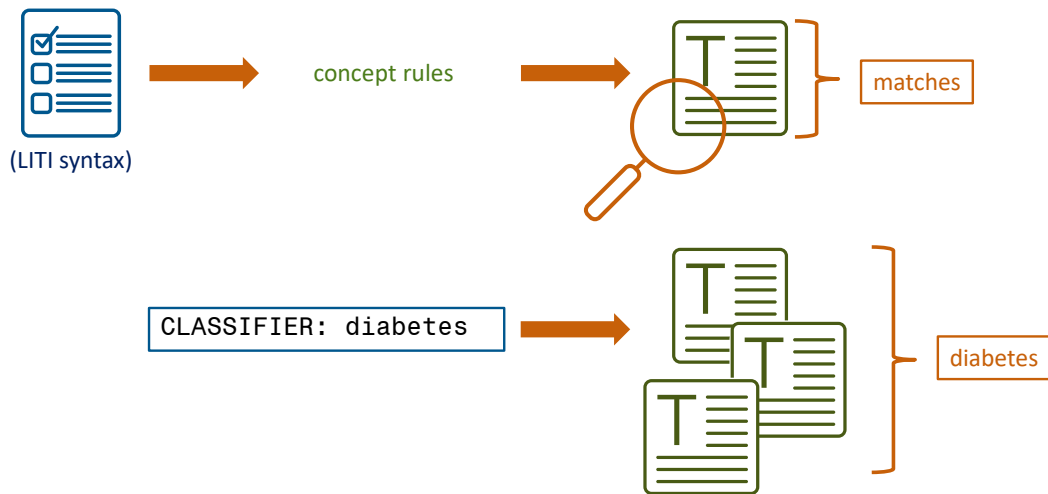
4.1 SAS Visual Text Analytics Rules



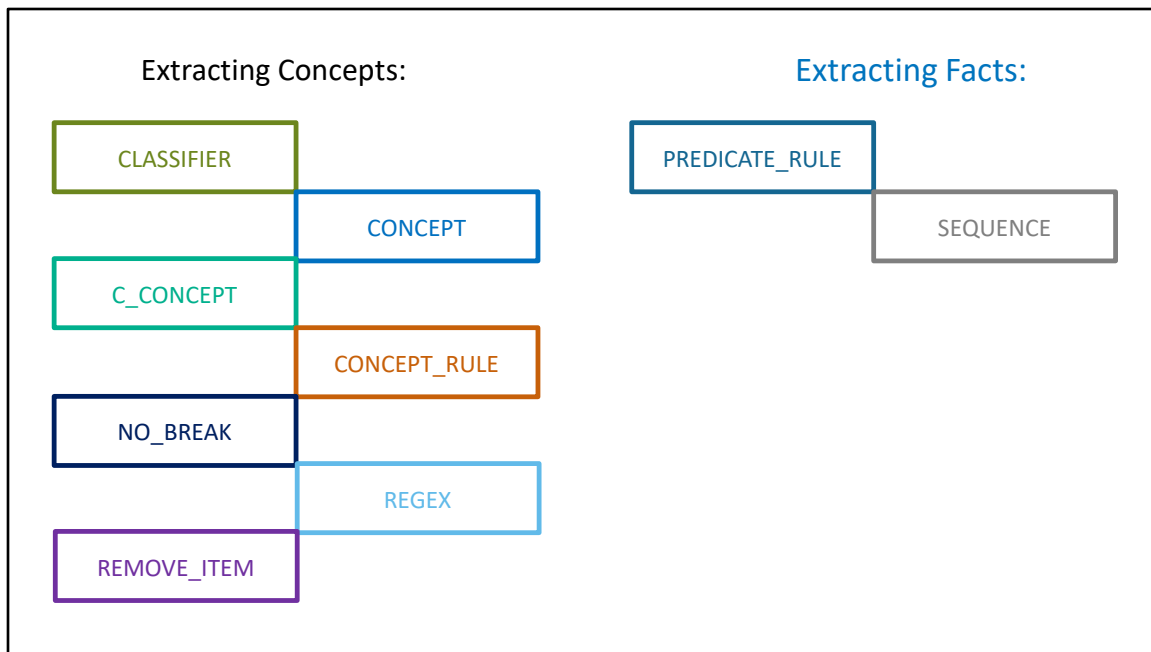
Overview of Concept Rules



Copyright © SAS Institute Inc. All rights reserved.



Concept rules are written using LITI (language interpretation and text interpretation) syntax. You write concept rules that recognize items in context so that you can extract only the pieces of the document that match the rule. For example, in the rule **CLASSIFIER: diabetes**, **CLASSIFIER** is the rule type and is written in uppercase letters, **diabetes** is the argument, and they are separated by a colon. The rule extracts all documents in your document collection that contain the word *diabetes*. Rule modifiers can be used to further refine the set of matches, because the rule syntax varies greatly depending on the rule type. Also, a single concept can reference one or more additional concepts.

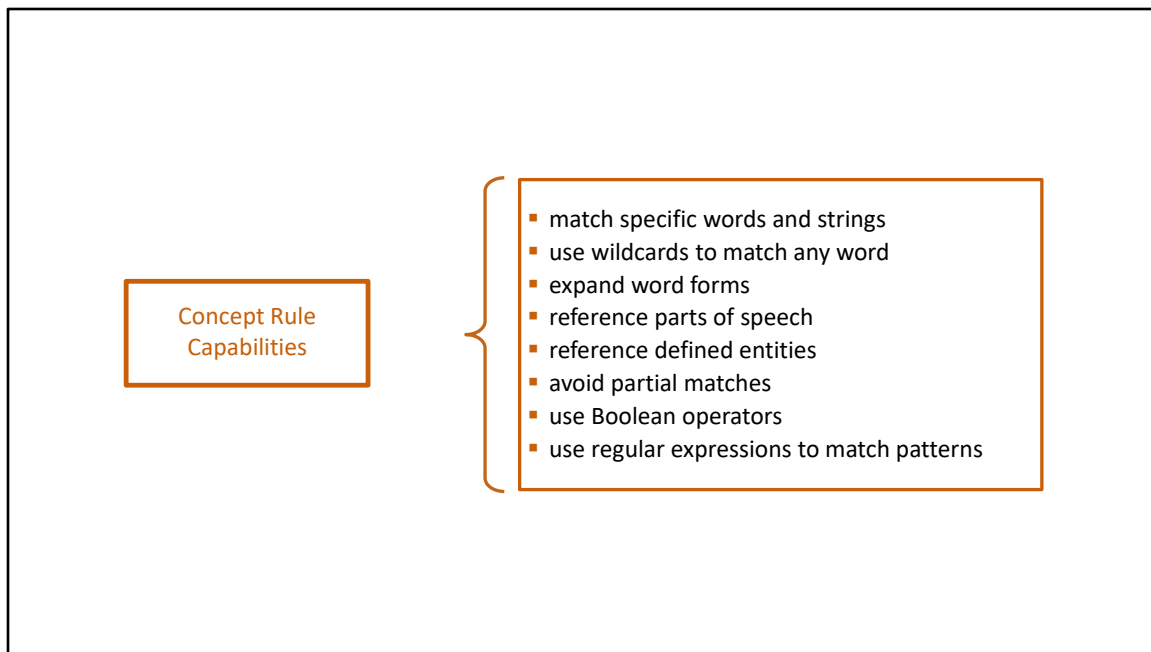


It's important to understand the rule types so that you can select those that efficiently generate the most matches for your purposes. The following rule types are used for extracting concepts: CLASSIFIER, CONCEPT, C_CONCEPT, CONCEPT_RULE, NO_BREAK, REGEX, and REMOVE_ITEM.

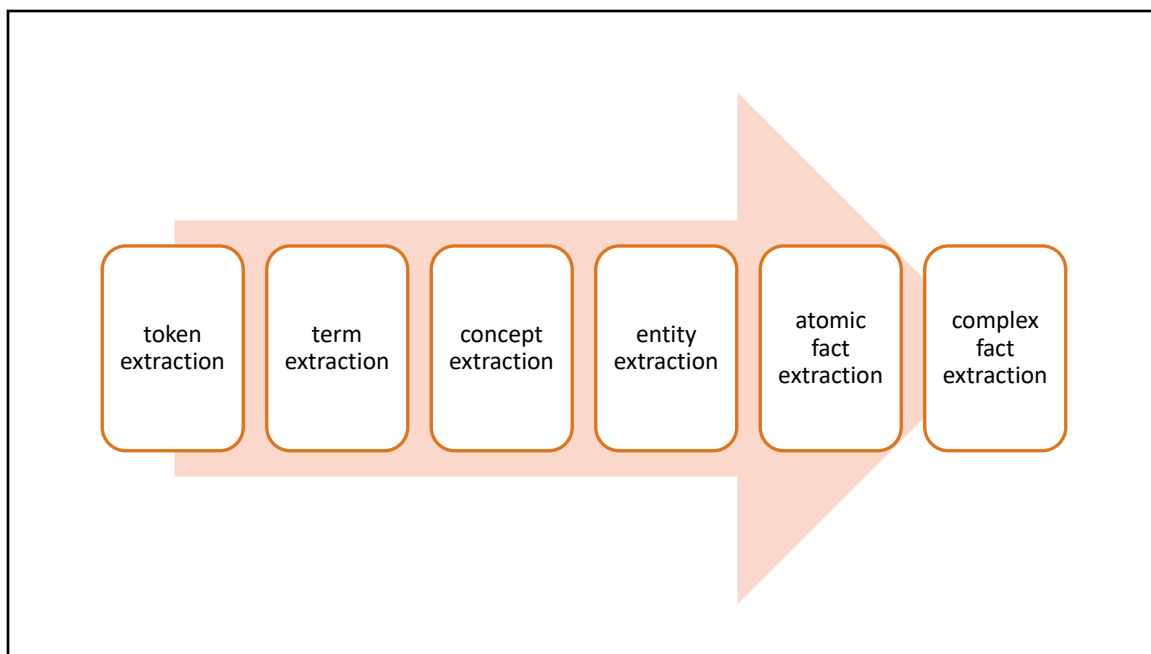
Facts (also called *predicates*) are related pieces of information that are located and matched together. For extracting facts, you can use the PREDICATE_RULE and SEQUENCE rule types.

A concept can be defined using more than one rule type.

For the purposes of this course, we aren't covering REMOVE_ITEM or SEQUENCE, but you can use the documentation to learn more about these rule types.



Each of the concept rule types provides one or more of the following capabilities: match specific words and strings, use wildcards to match any word, expand word forms, reference parts of speech, reference defined entities, avoid partial matches, use Boolean operators, and use regular expressions to match patterns.



As you move from one level of complexity to the next level, you engage additional natural language features of the software. Transitioning from terms to concepts and entities, you can benefit from predefined concepts, or you can exploit your domain expertise to add custom concepts.

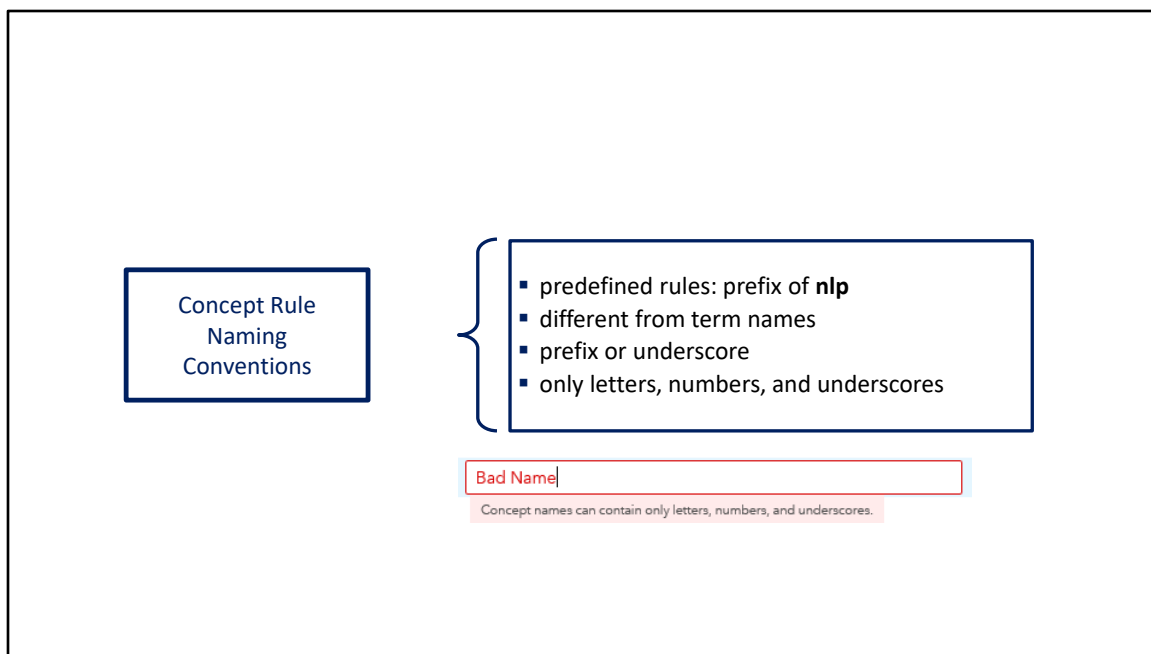
Parsing a document begins with token extraction. Tokens are just contiguous sets of characters separated by white space or punctuation. When you add language features, such as parts-of-speech, you increase the complexity from token extraction to term extraction. With more sophisticated natural language processing to identify concepts, such as noun phrases, measurements, or places, you can identify and extract more complex information from a document. Finally, by associating two or more concepts, you can extract facts or predicates, such as information that combines a university concept with a presidential concept to identify universities named after U.S. presidents. This is called atomic fact extraction.

Complex fact extraction is the goal of artificial intelligence, namely having computers that communicate directly with humans. Text analytics focuses on a specific document collection and avoids cumbersome overhead that would be needed to accommodate ad hoc queries spanning all human knowledge.

Concept Rule Naming Conventions



Copyright © SAS Institute Inc. All rights reserved.



Predefined concept rules use the prefix **nlp** for natural language processing. Concept names should not be the same as term names. You can use a prefix or an underscore to avoid ambiguities with terms. Concept names can contain only letters, numbers, and underscores. The example concept names we'll use all have at least one underscore to distinguish them from actual terms.

Concept Rule
Coding
Conventions

- **Punctuation to qualify matches:**
 - ~~CLASSIFIER, CONCEPT~~
- **Colon:**
 - separates rule types and tags: **CLASSIFIER:**
 - between arguments in PREDICATE_RULE
 - before part-of-speech tag: **:Prep**
- **Comma:**
 - separates arguments in rule definitions
 - separates logical operators in PREDICATE_RULE

You use punctuation to qualify the matches for all rule types except CLASSIFIER and CONCEPT.

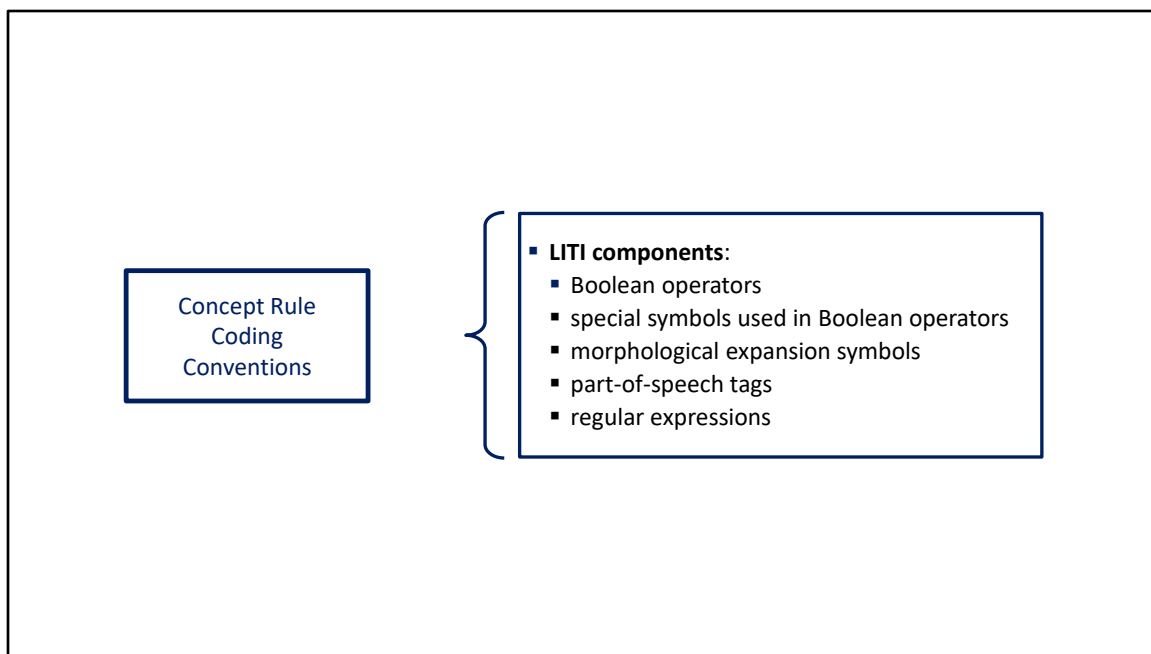
A colon separates rule types and tags after a concept rule type, for example CLASSIFIER:. You use a colon between arguments in a PREDICATE_RULE definition, and before a part-of-speech tag, for example, :Prep.

A comma separates elements such as arguments in a rule definition. You add a space after the comma and before the next element. Also, you can use a comma to separate logical operators in a PREDICATE_RULE definition.

Concept Rule Coding Conventions

- **Single space:**
 - separate strings, concept names, part-of-speech tags, rule modifiers
 - CONCEPT, CONCEPT_RULE, and C_CONCEPT
- **Quotation marks:**
 - enclose concept names and strings
 - CONCEPT_RULE, REMOVE_ITEM, PREDICATE_RULE
- **Parentheses:**
 - group elements
 - CONCEPT_RULE, REMOVE_ITEM, SEQUENCE, PREDICATE_RULE
 - Boolean operators (AND, OR, SENT, DIST_n, and ORDDIST_n)
- **Square braces:**
 - group elements
 - REGEX

You use a single space to separate strings, concept names, part-of-speech tags, and rule modifiers in CONCEPT, CONCEPT_RULE, and C_CONCEPT definitions. You use quotation marks to enclose concept names and strings in CONCEPT_RULE, REMOVE_ITEM, and PREDICATE_RULE definitions. You use parentheses to group the elements in CONCEPT_RULE, REMOVE_ITEM, SEQUENCE, and PREDICATE_RULE rule types, and within certain Boolean operators (AND, OR, SENT, DIST_n, and ORDDIST_n). And you use square braces to group elements in the REGEX rule type.



Other LITI components include Boolean operators, special symbols used in Boolean operators, morphological expansion symbols, part-of-speech tags, and regular expressions.

Additional Concept Rule Guidelines

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Boolean Operators for Extracting Concept Rules and Facts

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Special Symbols Used in Boolean Rules

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Morphological Expansion Symbols in Concept Rules

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Part-of-Speech Tags (for English)

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



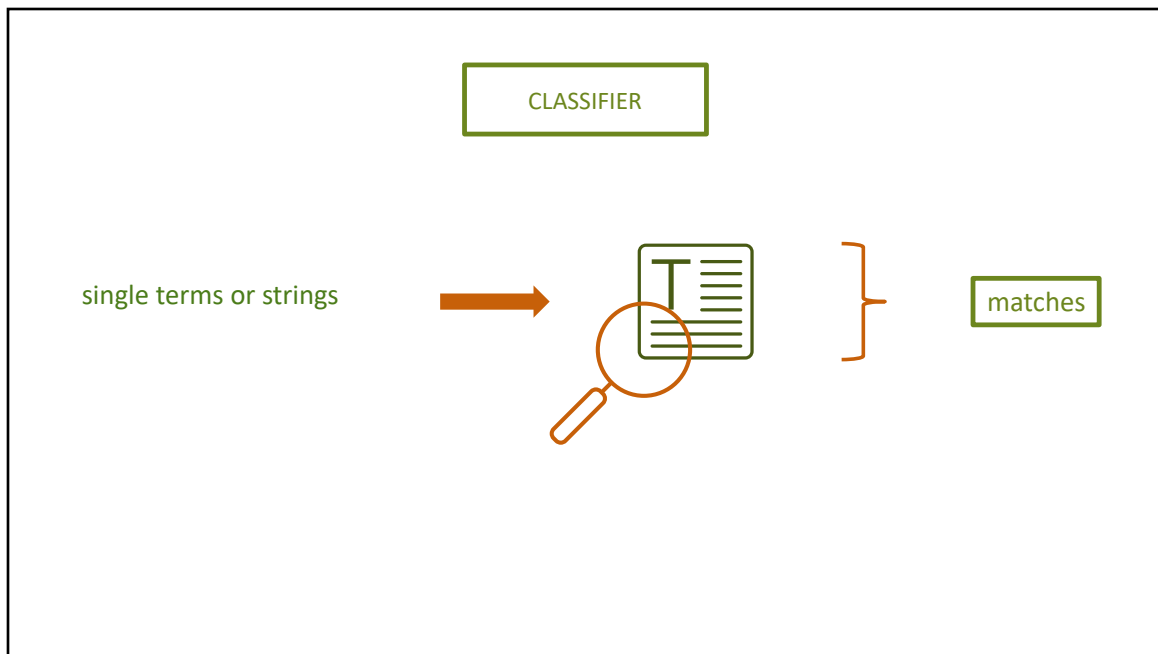
4.2 SAS Visual Text Analytics Concept Rules



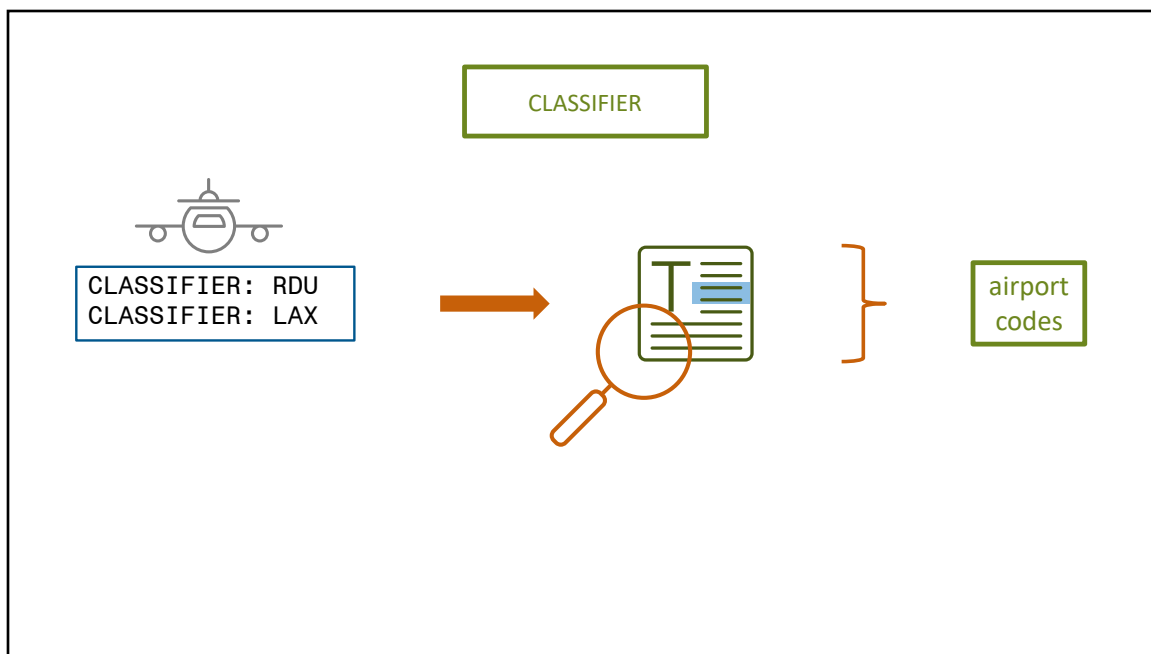
Exploring the CLASSIFIER Rule



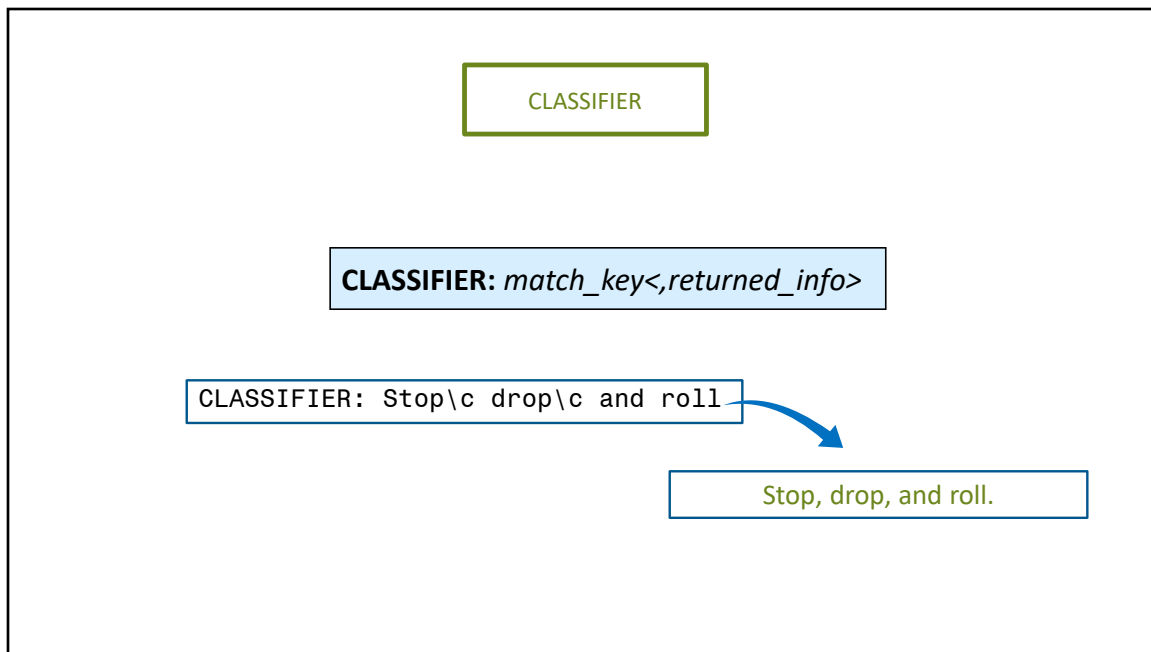
Copyright © SAS Institute Inc. All rights reserved.



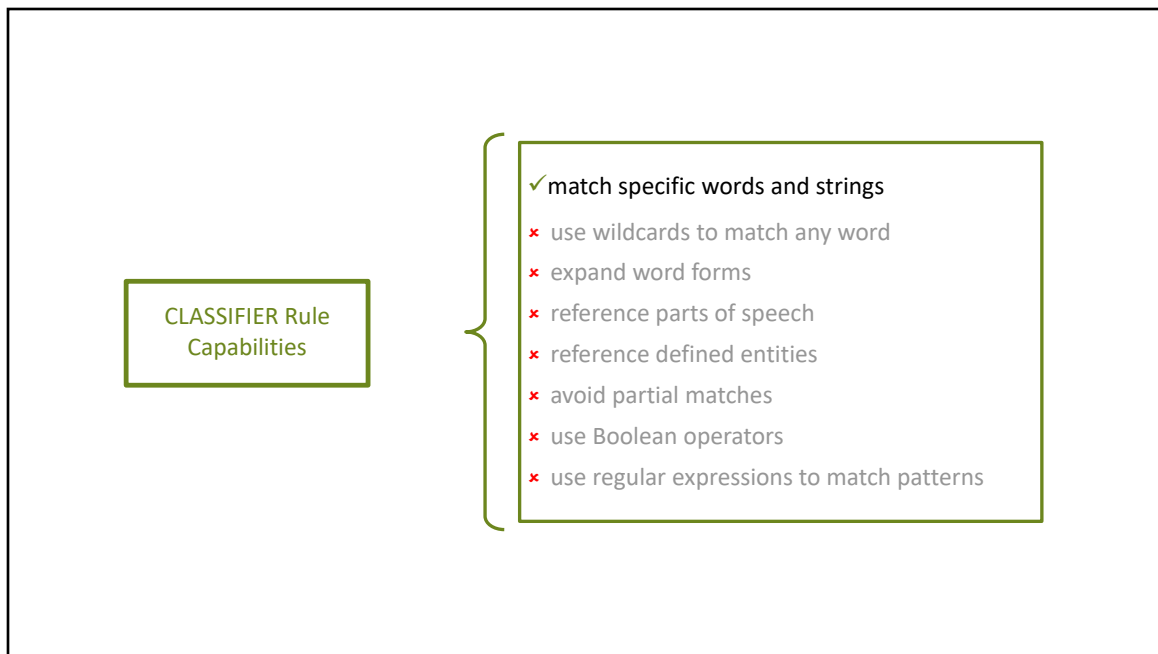
Let's start with the CLASSIFIER rule type. This rule identifies single terms or strings that you want to match in context.



For example, in a concept definition, you can create CLASSIFIER rules that contain specific airport codes. The portions of text that contain the airport codes are considered matches to the CLASSIFIER rules.



In the syntax, *match_key* is the literal string to match in the document. *Returned_info* is an optional string to return for the matched concept. You can omit the comma if you don't specify *returned_info*. You can use the character combination \c to match a comma within the *match_key*. For example, the sentence *Stop, drop, and roll.* is returned as a match for the rule **CLASSIFIER: Stop\c drop\c and roll.**



So out of the available concept rule type capabilities, the CLASSIFIER rule matches specific words and strings.

CLASSIFIER Rules for an Aviation Concept

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



CLASSIFIER Rule Examples



Copyright © SAS Institute Inc. All rights reserved.

CLASSIFIER

Rule	Example Matches
CLASSIFIER: Orion Glimmer C20	<i>Orion Glimmer C20</i> <i>orion glimmer c20</i> <i>ORION GLIMMER C20</i>
CLASSIFIER: very good value	<i>Very good value</i> <i>VERY good value</i> <i>very GOOD value</i>
CLASSIFIER: disappointing	<i>Disappointing</i> <i>disappointing</i>

Here are some examples of CLASSIFIER rules and their matches. Note that the matching is not case sensitive, and each CLASSIFIER rule corresponds to one term, or dictionary entry.

CLASSIFIER

Rule	Example Result
CLASSIFIER: [coref=drug]:Elevex	If the term <i>Elevex</i> appears at least once in the document, the term <i>drug</i> also returns matches for <i>Elevex</i> .
CLASSIFIER: [coref=author,writer,Larsson]: Stieg Larsson	If the string <i>Stieg Larsson</i> appears at least once in the document, the terms <i>author</i> , <i>writer</i> , and <i>Larsson</i> also return matches for <i>Stieg Larsson</i> .

Coreference, represented with the operator `coref=`, is a relationship between two words or phrases in which both refer to the same person or thing. So, in the first CLASSIFIER LITI definition, if the term *Elevex* appears at least once in the documents, the term *drug* also returns a match. In the example, three matches are identified: *Elevex*, *drug*, and *Elevex*. If the term *Elevex* did not appear, then no matches would be identified, and no information would be extracted.

In the second definition, if the string *Stieg Larsson* appears at least once in the document, the terms *author*, *writer*, and *Larsson* also return matches. A total of five matches are identified, so this example returns five rows of matches in the scored data set related to this concept, and possible other rows for other concepts.

CLASSIFIER

Rule	Example Result
CLASSIFIER: [export=AR:accounts receivable]:Sokolov	If a document matches <i>Sokolov</i> , then the CLIENT concept is flagged. If the document also contains <i>accounts receivable</i> , then the AR concept is flagged.

Additional modifiers are accommodated (for example, the EXPORT feature can be used to invoke a separate concept when a classifier term is in a document). Suppose the CLIENT concept is defined by this statement: **CLASSIFIER:[export=AR:accounts receivable]:Sokolov**. CLASSIFIER colon square bracket export equal AR colon accounts receivable close bracket Sokolov

If a document matches *Sokolov*, then the CLIENT concept is flagged. If the document also contains *accounts receivable*, then the AR concept is flagged.

Demo: Using the CLASSIFIER Rule

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: CLASSIFIER Rule



Copyright © SAS Institute Inc. All rights reserved.

Question: CLASSIFIER Rule

Which CLASSIFIER rule has the correct syntax to extract the text *SAS Visual Text Analytics*?

- a. CLASSIFIER: SAS Visual Text Analytics_C
- b. CLASSIFIER: SAS Visual Text Analytics
- c. CLASSIFIER: "SAS Visual Text Analytics"
- d. CLASSIFIER: ('SAS Visual Text Analytics')

Answer: CLASSIFIER Rule

Which CLASSIFIER rule has the correct syntax to extract the text *SAS Visual Text Analytics*?

- a. CLASSIFIER: SAS Visual Text Analytics _C
- ☒ b. CLASSIFIER: SAS Visual Text Analytics
- c. CLASSIFIER: "SAS Visual Text Analytics"
- d. CLASSIFIER: ('SAS Visual Text Analytics')

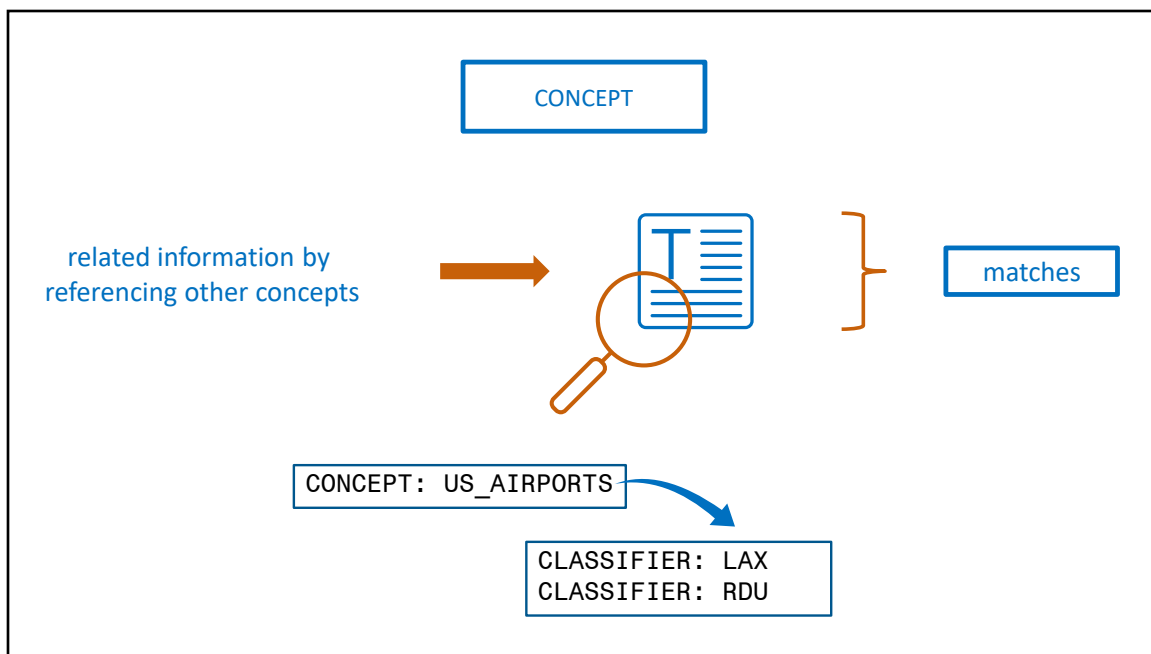
Answer: b

You can eliminate choice A because the CLASSIFIER rule does not support an _C marker. Choices C and D are eliminated because they would require matches on double quotes, parentheses, and single quotes. Choice B supplies the exact word sequence with no extraneous characters that would lead to a match for SAS Visual Text Analytics.

Exploring the CONCEPT Rule

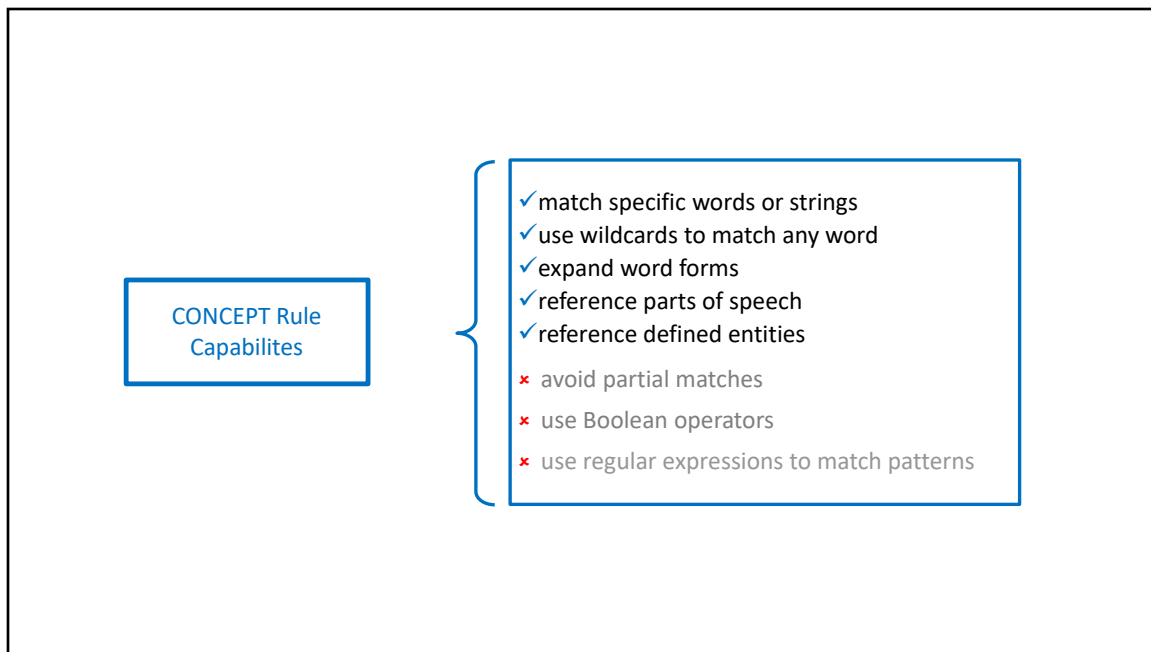


Copyright © SAS Institute Inc. All rights reserved.



Next we'll explore the CONCEPT rule. The CONCEPT rule identifies related information by referencing other concepts. For example, to capture documents that contain certain US airport names and locations, you can create a CONCEPT rule type in the definition. The CONCEPT rule type can reference any other concept. For example, it can reference a concept that contains a list of CLASSIFIER rules defining airport codes, thereby accessing a list of airport codes.

CONCEPT is a rule type. It's not to be confused with a "concept" in the general sense.



The capabilities of the CONCEPT rule include match specific words or strings, use wildcards to match any word, expand word forms, reference parts of speech, and reference defined entities.

CONCEPT

CONCEPT: *argument-1*<*argument-n*>

one space before:

- tokens
- concept names
- part-of-speech tags
- *_w* and *_cap* terms
- *_c*

In the syntax, *argument* can be a concept name, a rule modifier, or string. When you write CONCEPT definitions, enter at least one space before each of the following items: tokens (or literal strings), concept names, part-of-speech tags, *_w* and *_cap* terms, which are wildcards, and the context marker, *_c*, when preceded by a token, comma, or the name of a concept.

CONCEPT

Rule and Match Key	Example Matches
CONCEPT : SAS _w	SAS <i>employees</i> SAS <i>software</i>
CONCEPT : _w SAS	<i>Base</i> SAS <i>foundation</i> SAS
CONCEPT : SAS _cap	SAS <i>Institute</i> SAS <i>Analytics</i>

When you use the wildcard _w, the results match any word. So for example, if you specify **SAS _w**, the example matches include *SAS employees* and *SAS software* because they include any word that follows the word SAS. Or if you specify **_w SAS**, the example matches are any word followed by the word SAS, so *Base SAS* and *foundation SAS*.

When you use the wildcard _cap, the result matches any word that begins with an uppercase letter. So for example, if you specify **SAS _cap**, the example matches include *SAS Institute* and *SAS Analytics* because *Institute* and *Analytics* are both uppercase.

You specify both wildcards in lowercase.

CONCEPT

Symbol	Position	Result
@	Suffix	Stems the word that precedes the @ to include all forms of the word
@N	Suffix	Stems the word that precedes the @N to include noun forms of the word
@V	Suffix	Stems the word that precedes the @V to include verb forms of the word
_C	Suffix	Makes the rule case-sensitive

Here are some examples of additional arguments for CONCEPT rules. Three of these relate to expanding word forms. These three use morphological expansion symbols. Using the @ symbol stems the word that precedes the symbol to include **all forms** of the word. Using @N stems the word that precedes the @N to include **noun forms** of the word. Using @V stems the word that precedes the @V to include **verb forms** of the word, and using _C makes the rule **case-sensitive**.

CONCEPT

Rule	Example Matches
CONCEPT : go @	I will <i>go</i> . She is <i>going</i> . They <i>went</i> . He is <i>gone</i> .
CONCEPT : break @	It was <i>broken</i> . The camera <i>broke</i> . It kept <i>breaking</i> down. Give me a <i>break</i> !

So for example, if you specify **go @**, the example matches include all forms of the word *go*: *go*, *going*, *went*, and *gone*. And if you specify **break @**, the example matches includes all forms of the word *break*: *broken*, *broke*, *breaking*, and *break*.

CONCEPT

Part of Speech	Tag	Grammar Definition	Example Matches
Noun	N	SAS :N	SAS <i>employee</i> SAS <i>software</i>
Preposition	Prep	:Prep the drug	<i>of</i> the drug <i>from</i> the drug
Possessive Determinant	PossDet	:PossDet symptoms	<i>my</i> symptoms <i>her</i> symptoms <i>their</i> symptoms

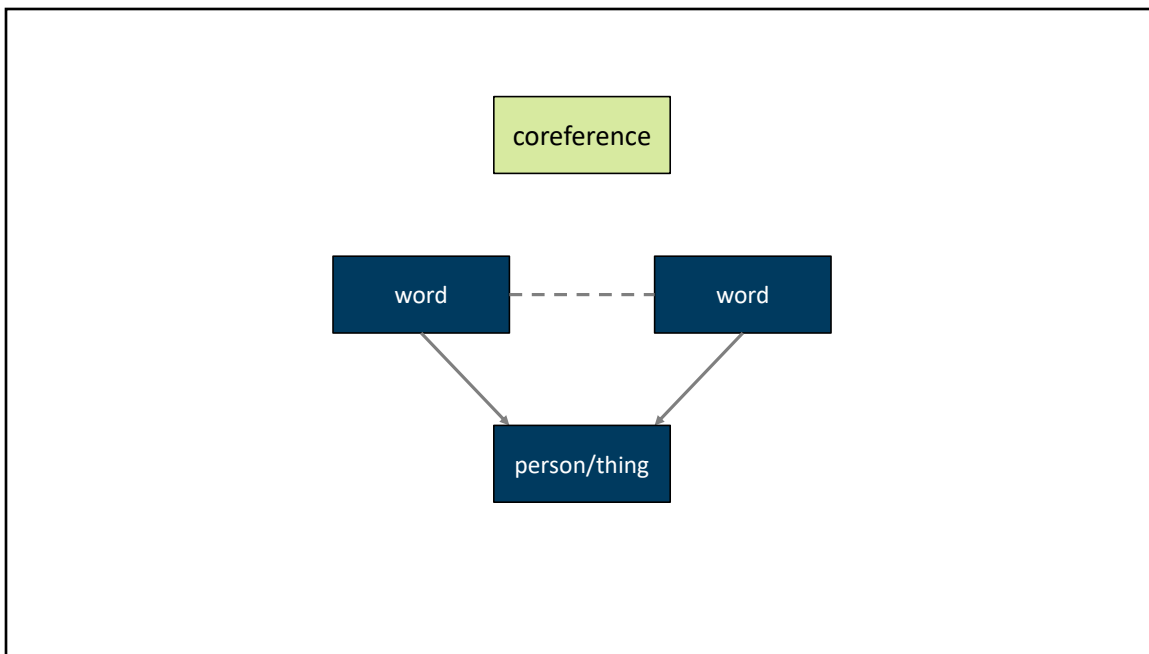
You precede part-of-speech tags with a colon when writing the definition. As an example, if you specify **:Prep the drug**, example matches include *of the drug* and *from the drug* because *of* and *from* are both prepositions.

Part-of-speech tags are case-sensitive, and you must type them in the case specified in the User's Guide.

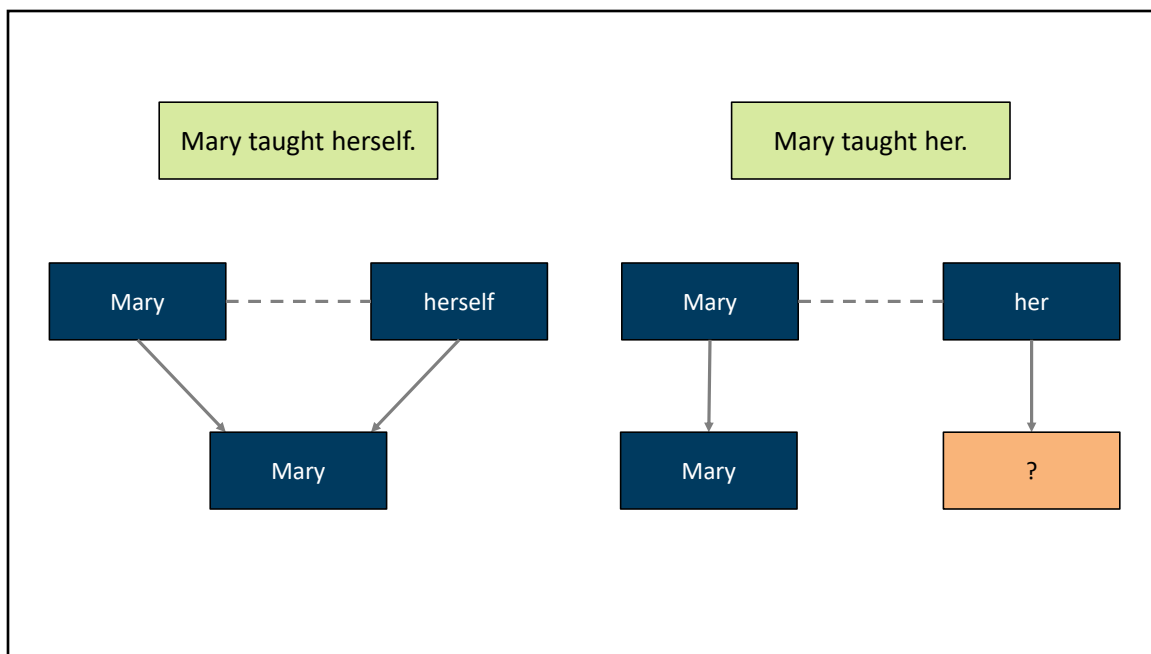
Coreference in the CONCEPT Rule



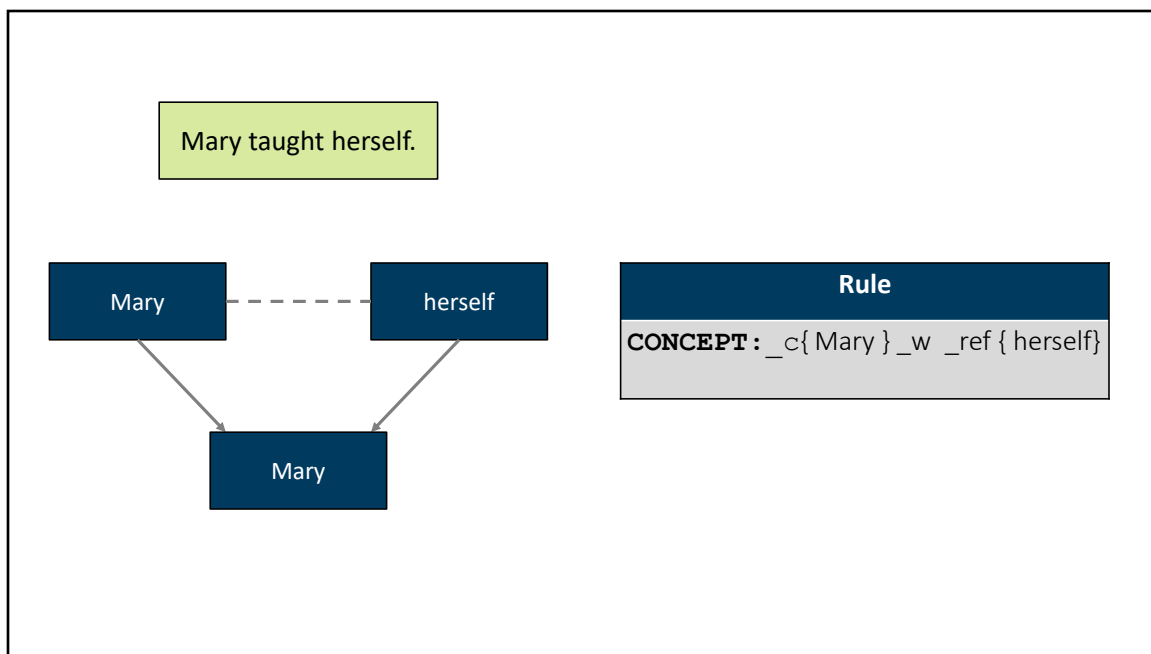
Copyright © SAS Institute Inc. All rights reserved.



Recall that coreference is a relationship between two words or phrases in which both refer to the same person or thing.



For example, in the phrase *Mary taught herself*, both the terms *Mary* and *herself* refer to the same person. However, in the phrase *Mary taught her*, the terms *Mary* and *her* refer to different people.



You can specify a canonical form and a referring term in a CONCEPT definition in a way that's similar to that in a CLASSIFIER definition. In the sentence *Mary taught herself*, *Mary* is the canonical form and *herself* is the referring term.

The canonical form is expressed in braces following the _c marker, and the referring term is expressed in braces following the _ref operator. You specify the _c and _ref operators in lowercase.

The definition can also include other items that are valid for the given definition type, such as literal text, part-of-speech tags, and references to other concepts.

Symbol	Result
>	Match all occurrences of the referring term
_F	Match all occurrences of the referring term that follow the canonical form
_P	Match all occurrences of the referring term that precede the canonical form

You can add symbols after the `_ref` operator to match additional occurrences of the referring term. The greater than symbol matches all occurrences of the referring term. `_F` matches all occurrences of the referring term that follow the canonical form. And `_P` matches all occurrences of the referring term that precede the canonical form.

Rule	Example Result
CONCEPT: _c{ doctor } :sep _ref { she }>	When I called my <i>doctor</i> , <i>she</i> said I should come in as soon as possible. At my next appointment, <i>she</i> changed my prescription.

Also, the text :sep is the part-of-speech tag for separator characters, including the period, comma, colon, and semicolon. In this example, the results match the term *doctor*, and both occurrences of *she*, the referring term. The second occurrence of *she* is matched because of the greater than (>) modifier.

Demo: Using the CONCEPT Rule

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: CONCEPT Rule



Copyright © SAS Institute Inc. All rights reserved.

Question: CONCEPT Rule

Which CONCEPT rule has the incorrect syntax to extract the text *SAS Visual Text Analytics*?

- a. CONCEPT: SAS Visual Text Analytics
- b. CONCEPT: SAS Visual _cap _cap
- c. CONCEPT: "SAS Visual Text Analytics"
- d. CONCEPT: _w Visual Text Analytics

Answer: CONCEPT Rule

Which CONCEPT rule has the incorrect syntax to extract the text *SAS Visual Text Analytics*?

- a. CONCEPT: SAS Visual Text Analytics
- b. CONCEPT: SAS Visual _cap _cap
- ☒ c. CONCEPT: "SAS Visual Text Analytics"
- d. CONCEPT: _w Visual Text Analytics

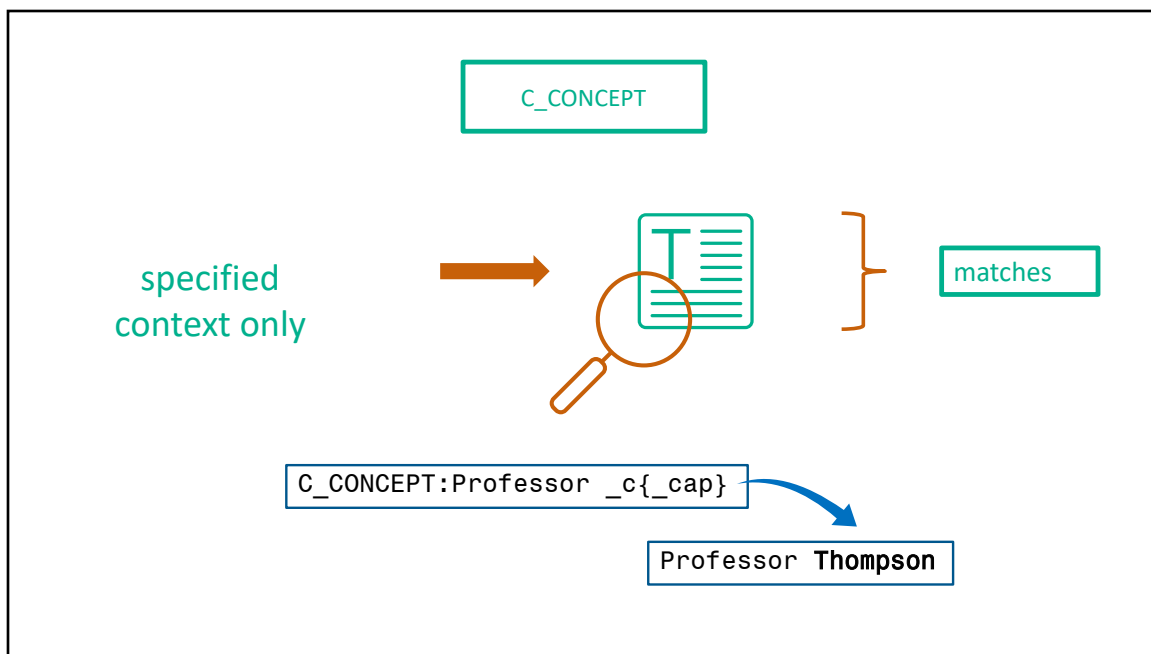
Answer: c

A CONCEPT rule without modifiers and special keywords behaves like a CLASSIFIER rule. Thus, the quotes in the rule would require quotes to appear in the matching string. All other options lead to a match. Option A is identical to a CLASSIFIER rule. Option B expands _cap to be a word that begins with an uppercase letter. Option D expands _w to match any word. If you were trying to match **SAS visual text analytics**, then options A and D would provide a match, but B would not because in "text analytics", neither word begins with an uppercase letter.

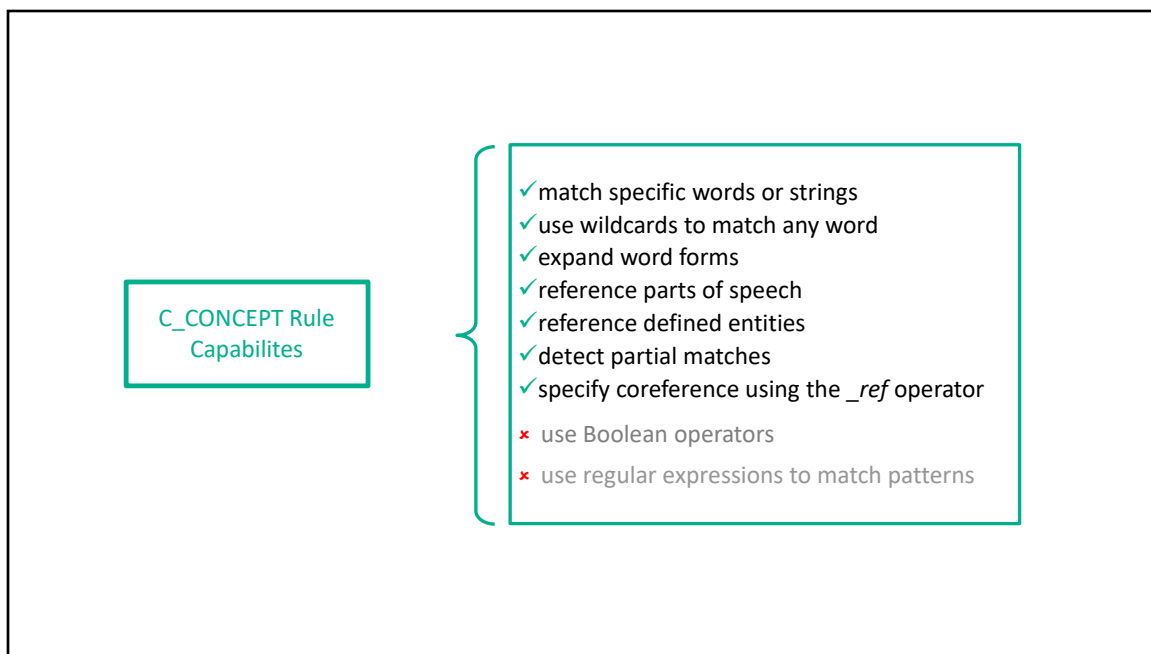
Exploring the C_CONCEPT Rule



Copyright © SAS Institute Inc. All rights reserved.



The `C_CONCEPT` rule returns matches that occur only in the specified context. For example, to extract matches that include names of university professors, you could create a `C_CONCEPT` rule that identifies matches on a concept (previously defined) that identifies last names only when the matched names are preceded by the word *Professor*.



The C_CONCEPT rule has all the capabilities of the CONCEPT rule plus the ability to match concepts with a context and detect partial matches. You specify coreference using the *_ref* operator.

C_CONCEPT

C_CONCEPT: <argument> _c{argument}<argument>

In the syntax, the context marker `_c` is required, and you can only specify one. It's used to locate related terms within a specific context. You specify the marker in lowercase, and the text or item to be matched is enclosed in braces. *Argument* can be a concept name, a rule modifier, or a string.

C_CONCEPT

Rule	Example Results
C_CONCEPT : New York _c { _cap _cap }	New York <i>Stock Exchange</i> New York <i>Public Library</i> New York <i>Daily News</i> New York theater district
C_CONCEPT : _c { _cap } Sea	<i>Bering</i> Sea <i>North</i> Sea <i>Baltic</i> Sea Bering Straight

Here are some examples of C_CONCEPT LITI rules and their matches. The first rule matches any two-word proper nouns that follow *New York* if the first letter of each word is uppercase. The second concept matches any word that begins with an uppercase letter followed by the word *Sea*.

C_CONCEPT

Rule Body	Location Specified	Example Results
Orion _c{Glimmer} C20	Definition of the product camera	Orion <i>Glimmer</i> C20
_c{I love} my camera	Tonal keyword, positive node	<i>I love</i> my camera
Auto Focus is _c{not fast enough}	Product definition, negative node for the focus feature	Auto Focus is <i>not fast enough</i>

The Rule Body shows the context in which information is identified and extracted. For the first rule, when *Glimmer* appears between *Orion* and *C20*, it is matched, and the term *glimmer* is extracted when scoring the document.

The second rule looks for a positive judgment associated with the specific keywords *my* and *camera*, and if found, returns the positive expression.

The third example looks for a negative assessment in context and extracts the negative expression when the context occurs.

C_CONCEPT

Rule	Example Results
C_CONCEPT : New York _c {Daily News }>	The <i>Daily News</i> of New York City is the fourth most widely circulated daily newspaper in the United States with a daily circulation of 605,677, as of November 1, 2011. The <i>Daily News</i> was founded by Joseph Medill Patterson in 1919. It was not connected to an earlier <i>New York Daily News</i> , which had been founded in the 1850s.

You can use the greater than symbol with the _c marker to specify a partial match. Every occurrence of the bracketed term is a match if the entire rule appears at least once in the input text. The example rule causes a match of *Daily News* in the context of being preceded by *New York*, and if *New York Daily News* appears at least one time in the document, then all occurrences of *Daily News* are matched, whether they appear before or after the initial match of *New York Daily News*.

Demo: Using the C_CONCEPT Rule

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: C_CONCEPT Rule



Copyright © SAS Institute Inc. All rights reserved.

Question: C_CONCEPT Rule

The concept rule C_CONCEPT: SAS _c{enterprise miner} is validated using the following test text: SAS Enterprise Miner is a solution to create accurate predictive and descriptive models on large volumes of data across different sources in the organization. Identify the text string that should be highlighted when you validate the rule.

- a. SAS Enterprise Miner
- b. Enterprise Miner
- c. SAS
- d. none of the above

Answer: C_CONCEPT Rule

The concept rule C_CONCEPT: SAS _c{enterprise miner} is validated using the following test text: SAS Enterprise Miner is a solution to create accurate predictive and descriptive models on large volumes of data across different sources in the organization. Identify the text string that should be highlighted when you validate the rule.

- a. SAS Enterprise Miner
- ☒ b. Enterprise Miner
- c. SAS
- d. none of the above

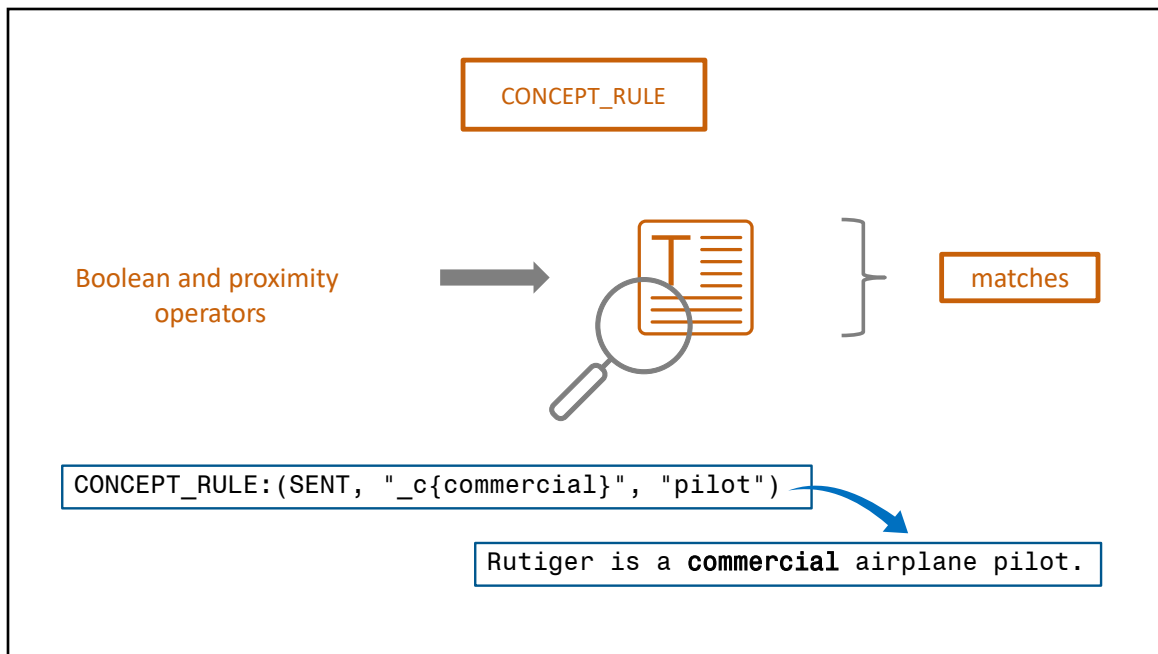
Answer: b

Answers A and C are incorrect because only the arguments in the _c marker will be matched, and the word SAS does not appear in the _c marker argument. B is correct because the _c marker contains the words Enterprise Miner. D is incorrect because B is correct.

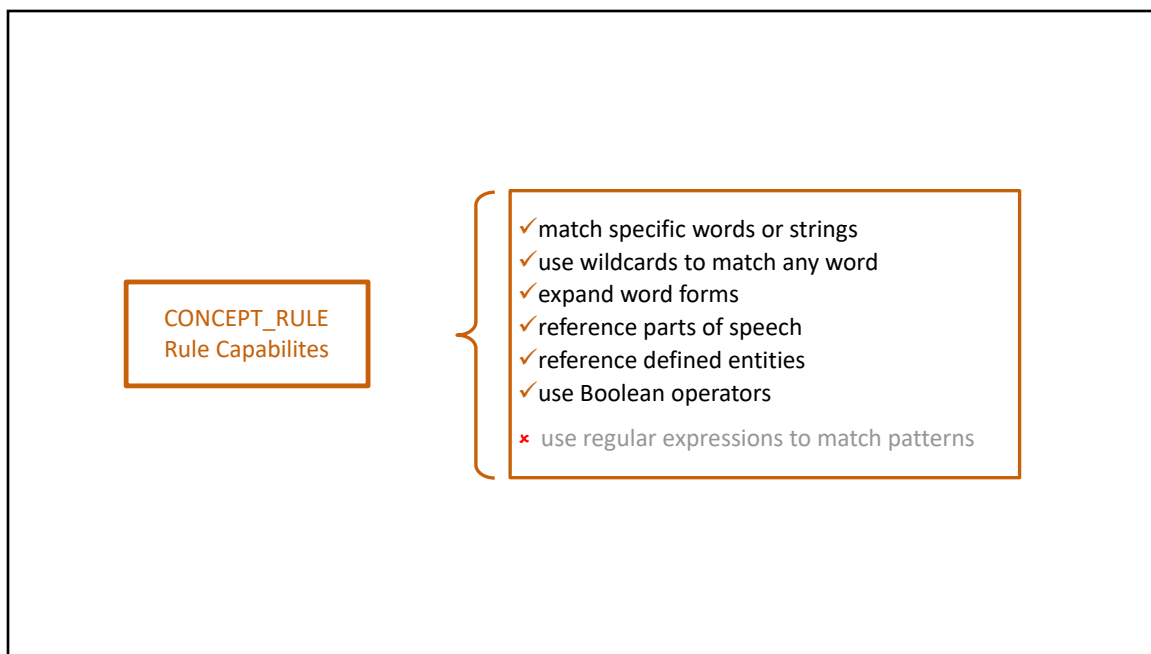
Exploring the CONCEPT_RULE Rule



Copyright © SAS Institute Inc. All rights reserved.



The CONCEPT_RULE rule uses Boolean and proximity operators to determine matches. In the example, if the word *commercial* appears in the same sentence as *pilot*, *commercial* is extracted from the document as information about a type of pilot.



The CONCEPT_RULE rule type has all the features of a C_CONCEPT rule, plus the ability to use Boolean operators to determine context matches.

CONCEPT_RULE

CONCEPT_RULE: (<Boolean-rule-1>...<Boolean-rule-n>)

(Boolean-operator, "_c{argument-1}",<"argument-2">...<"argument-n">)

The syntax is Boolean-rule-1, and then optional nested Boolean rules 2 thru n. At least one Boolean operator is required. A Boolean-operator is applied to arguments written as "argument-1", "argument-2", and so on. Arguments must be enclosed in double quotation marks, and there must be a space or blank before each argument. Exactly one argument must be an _c marker argument, unless a logical OR condition is specified, then more than one _c marker is allowed. The _c marker does NOT have to be the first argument, but it must be positioned so that when the rule resolves to TRUE, the _c marker term will exist in the document and will be extracted. Consequently, an _c marker should never appear in a NOT condition.

The other arguments that appear in quotation marks provide context for the match and must be present or absent as specified for a match to occur.

CONCEPT_RULE

Concept Name	Rule
<i>YearCpt</i>	REGEX : \d\d\d\d
<i>BirthYear</i>	CONCEPT_RULE : (SENT, "born", "_c{YearCpt}")

Example Results

Charles Dickens was *born* in a modest home in Portsmouth, England, in **1812**.
J.K. Rowling, *born* **1965**, is best known as the author of the Harry Potter series.

In this example, the **BirthYear** concept uses a **CONCEPT_RULE** definition that references the **YearCpt** concept. **YearCpt** matches exactly four contiguous numeric digits, so unusual years are possible, such as 2525. The Boolean operator **SENT** causes the **BirthYear** concept to extract the **YearCpt** year when it appears in the same sentence as the word *born*. Although the word *born* is highlighted in the Example Results to illustrate how the rule works, only the year is matched because it is the argument to the **_c** marker.

Demo: Using the `CONCEPT_RULE` Rule

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: CONCEPT_RULE



Copyright © SAS Institute Inc. All rights reserved.

Question: CONCEPT_RULE

Which CONCEPT_RULE body has the correct syntax?

- a. Orion _cap _c{is wonderful}
- b. (AND, Orion _cap, _c{is wonderful})
- c. (SENT, "OrionGlimmerprice", "_c{affordable}")
- d. (SENT, "OrionGlimmerprice", "affordable")

Answer: CONCEPT_RULE

Which CONCEPT_RULE body has the correct syntax?

- a. Orion _cap _c{is wonderful}
- b. (AND, Orion _cap, _c{is wonderful})
- ☒ c. (SENT, "OrionGlimmerprice", "_c{affordable}")
- d. (SENT, "OrionGlimmerprice", "affordable")

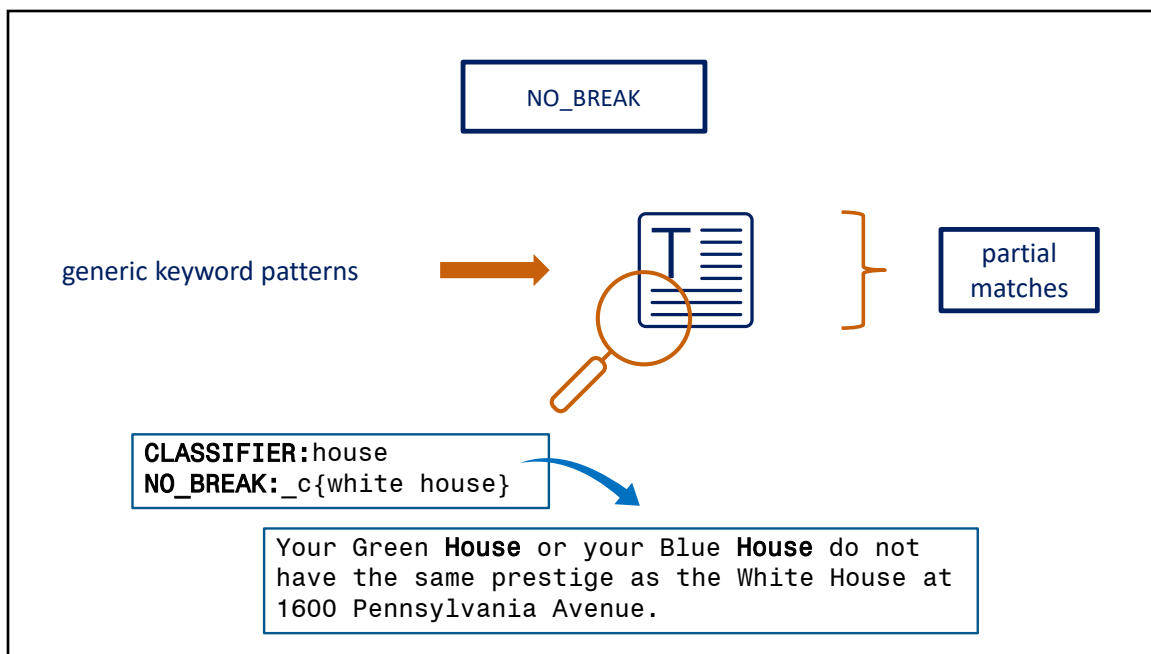
CONCEPT_RULE rules must include at least one Boolean operator and one _c marker. Arguments to Boolean operators are specified in double quotation marks.

Correct answer: c

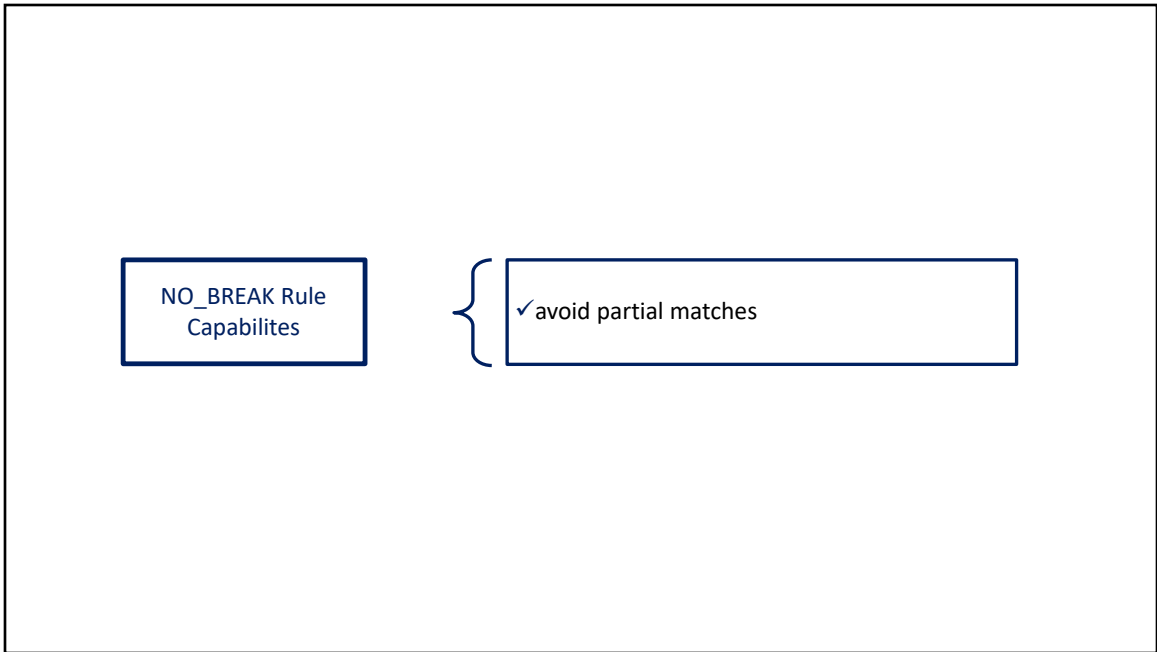
Exploring the NO_BREAK Rule



Copyright © SAS Institute Inc. All rights reserved.



The NO_BREAK rule prevents partial matches for generic keyword patterns. In this example, references to *house* are matched and extracted unless the reference is to the presidential residence, the *White House*.



The only capability of the NO_BREAK rule, therefore, is to avoid partial matches.

NO_BREAK

NO_BREAK: *_c{argument}*

- applies across entire taxonomy
- insert all in one concept
- globally implemented rules:
 - NO_BREAK
 - REMOVE_ITEM
- troubleshoot matching behavior

In the syntax, the *_c* marker is required and is lowercase. It can surround only one argument, and the argument can be a concept or a string.

There are some special considerations for the NO_BREAK rule. NO_BREAK applies across the entire taxonomy, regardless of where the rule appears or whether the rule is enabled or disabled.

You should not insert NO_BREAK rules just anywhere. It's helpful to insert them all in one concept. That is, create a concept that contains globally implemented rules only (NO_BREAK or REMOVE_ITEM). Having such rules all in one place aids in troubleshooting the matching behavior across your taxonomy.

NO_BREAK

Concept Name	Rule
MAX_LIKE	CLASSIFIER: maximum likelihood
OTHER_LIKE	CLASSIFIER: likelihood NO_BREAK: _c{MAX_LIKE}

Example Results

Ordinary gamma models are fit using *maximum likelihood* estimation.
Survival models obtain estimates using a partial *likelihood* formulation.

In this example, the **OTHER_LIKE** concept uses a NO_BREAK definition that references the **MAX_LIKE** concept. The term *likelihood* is matched unless it satisfies the rules for the **MAX_LIKE** concept. The example features two documents. The first document matches the **MAX_LIKE** concept. The second document has terms to match the **OTHER_LIKE** concept.

Demo: Using the NO_BREAK Rule

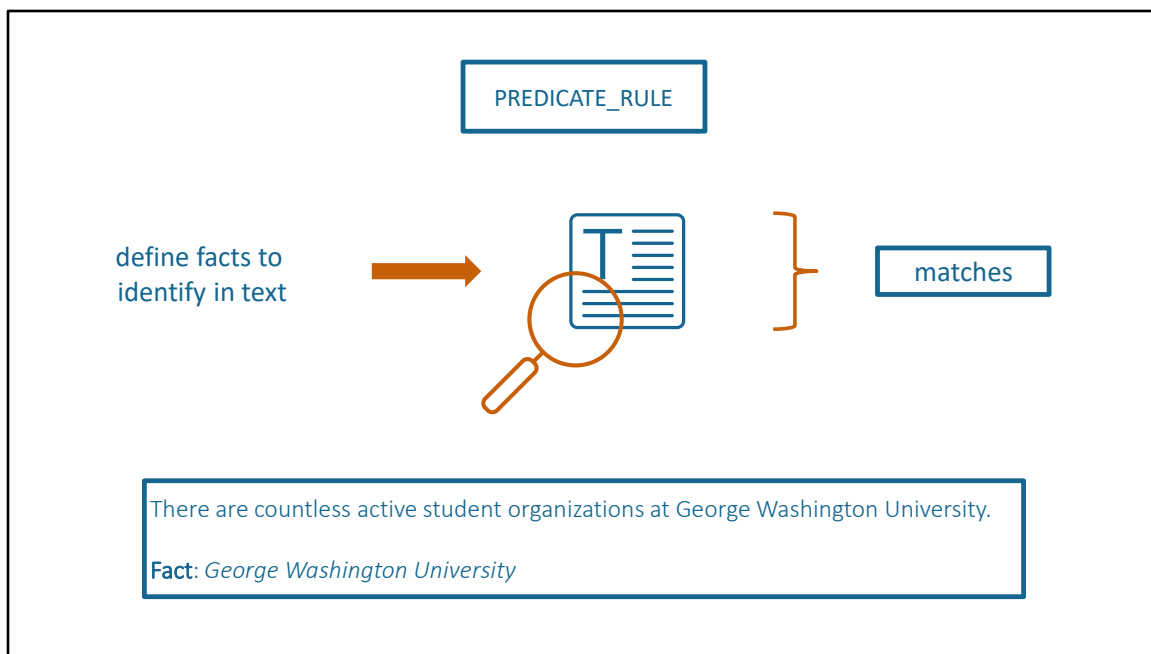
To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Exploring the PREDICATE_RULE Rule

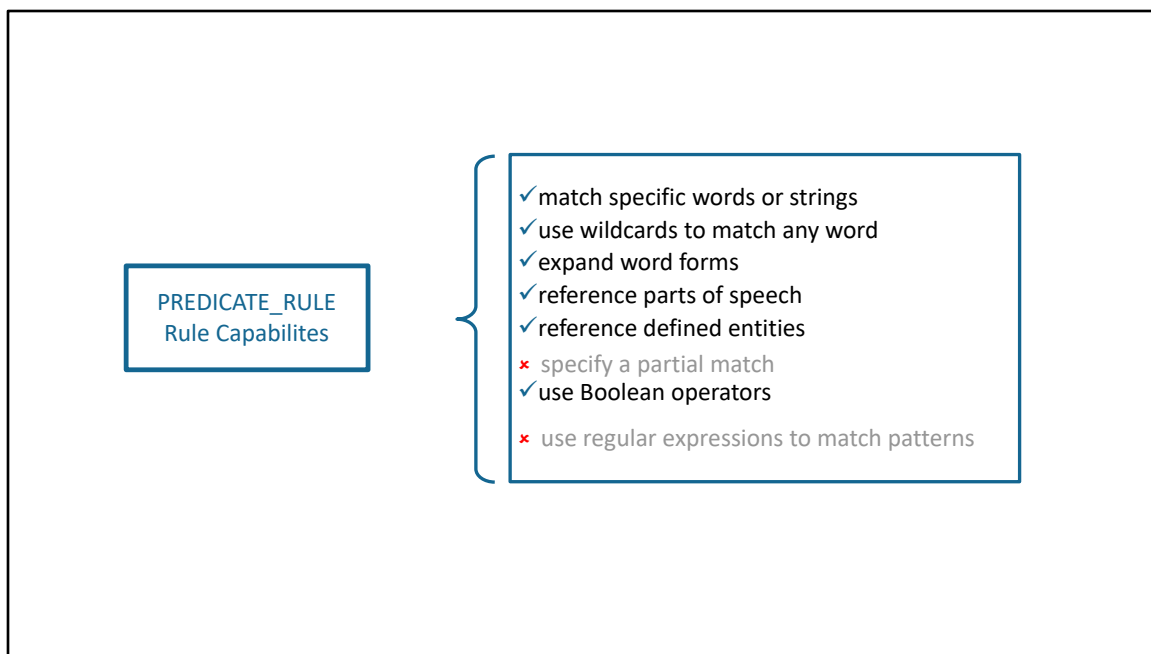


Copyright © SAS Institute Inc. All rights reserved.



You can use the **PREDICATE_RULE** rule to define facts that you want identified in text. Facts, which are also called *predicates*, are related pieces of information that are located and matched together.

For example, *George Washington University* can be considered as a fact in the following sentence: *There are countless active student organizations at George Washington University.*



The PREDICATE_RULE rule can match specific words or strings, use wildcards to match any word, expand word forms, reference parts of speech and defined entities, and use Boolean operators.

PREDICATE_RULE

```
PREDICATE_RULE: (arg1<,arg2,arg3,...>):  
                (Boolean_Op, "_arg1{match_key}",  
                <,"_arg2{match_key}", "_arg3{match_key}"...>)
```

In the syntax, at least one Boolean operator is required, and you must specify at least one argument, such as **arg1**. Arguments must be named using lowercase letters and digits and begin with a lowercase letter. The `_c` marker is not allowed in the `PREDICATE_RULE` type, but you can specify multiple markers. This enables you to define arguments that locate relationships between two or more concepts. When multiple markers are included in the rule, the match is highlighted from the first marker to the last marker.

Match_key can include Boolean operators, literals, part-of-speech tags, wildcards, and references to other concepts.

PREDICATE_RULE

Concept Name	Rule
<i>CountryNames</i>	CLASSIFIER: France CLASSIFIER: French CLASSIFIER: India CLASSIFIER: Indian CLASSIFIER: Canada ...
<i>OfficialTitles</i>	CLASSIFIER: Prime Minister CLASSIFIER: President
<i>WorldLeaders</i>	PREDICATE_RULE: (name,title,co): (SENT, "_name{_cap _cap}", "_title{OfficialTitles}", "_co{CountryNames}")

In this example, the **WorldLeaders** concept uses a **PREDICATE_RULE** definition that references the concept **OfficialTitles** and the concept **CountryNames**. An additional argument **name** references two words beginning with uppercase letters. Every instance of these three arguments is obtained, and all words in between the first and last occurrence of any argument, inclusive, is matched if all three argument types appear within the same sentence.

PREDICATE_RULE

Rule	Example Matches
PREDICATE_RULE: (aa,bb): (SENT, "_aa{good}", "_bb{quality}")	The <i>quality is good</i> . It's very <i>good quality</i> .
PREDICATE_RULE: (one,two): (AND, "_one{value}", "_two{price}")	It's a good <i>value for the price</i> . Couldn't believe the <i>price</i> . <i>What a value!</i>
PREDICATE_RULE: (xx,yy): (DIST_3, "_xx{not}", "_yy{disappointed}")	You will <i>not be disappointed</i> . I was <i>not at all disappointed</i> .

In these examples, the first PREDICATE_RULE looks for the words *good* and *quality* in the same sentence and matches these words and all words in between when the Boolean SENT operator is satisfied.

The second PREDICATE_RULE finds the words *value* and *price* when both occur in the document and matches the two specified words and all words in between.

The third PREDICATE_RULE uses a Boolean DIST_3 operator looking for *not* and *disappointed* within three words of each other.

PREDICATE_RULE in SAS Visual Text Analytics

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Using the PREDICATE_RULE Rule

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: PREDICATE_RULE



Copyright © SAS Institute Inc. All rights reserved.

Question: PREDICATE_RULE

Which PREDICATE_RULE body has the correct syntax?

- a. (aa,bb): (DIST_5, '_aa{cameraprice}', '_bb{expensive}')
- b. (yy,zz): (DIST_5, _yy{"cameraprice"}, _zz{"expensive"})
- c. (xx,yy): (DIST_5, "_xx{cameraprice}", "_yy{expensive}")
- d. (aa,bb): (DIST_5,_aa{cameraprice},_bb{expensive})

Answer: PREDICATE_RULE

Which PREDICATE_RULE body has the correct syntax?

- a. (aa,bb): (DIST_5, '_aa{cameraprice}', '_bb{expensive}')
- b. (yy,zz): (DIST_5, _yy{"cameraprice"}, _zz{"expensive"})
- ☒ c. (xx,yy): (DIST_5, "_xx{cameraprice}", "_yy{expensive}")
- d. (aa,bb): (DIST_5,_aa{cameraprice},_bb{expensive})

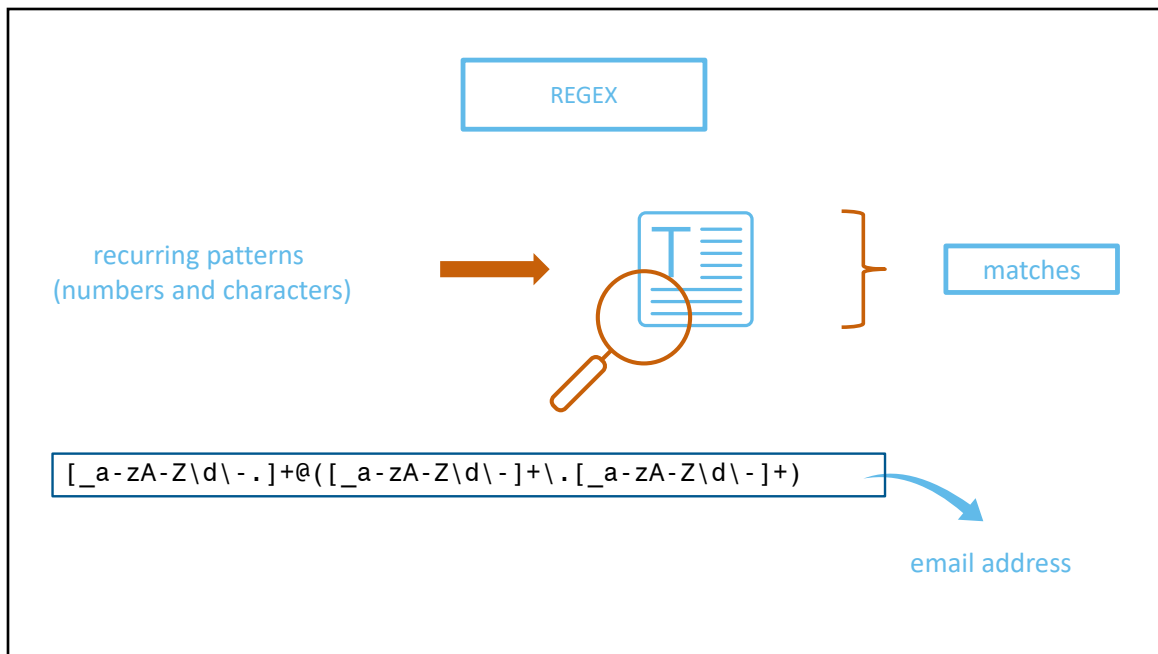
Boolean operators are followed by a comma and a space.
Arguments are specified in double quotation marks and
are separated by a comma and a space.

Correct answer: c

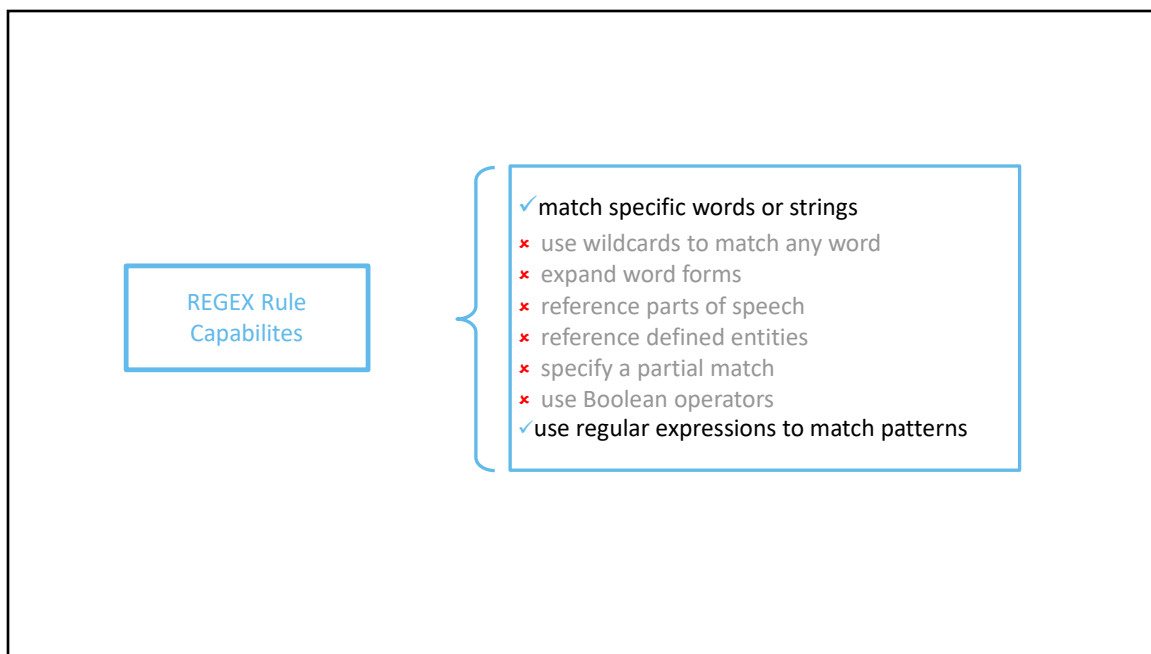
Exploring the REGEX Rule



Copyright © SAS Institute Inc. All rights reserved.



The REGEX rule identifies recurring patterns of information in text that can be expressed in numbers and characters, such as telephone numbers, license plate numbers, part numbers for manufacturing components, hyphenated words, and so on. Use regular expressions when you want to locate matches on text that has recognizable patterns, such as email and street addresses.



The REGEX rule capabilities include match specific words or strings, and use regular expressions to match patterns.

REGEX

- elements grouped using []
- meta characters, [], (,), ?, *, +, ., \, |, and \$, must be preceded with a backslash
 - [\?] returns ?
- cannot refer to concepts
- match a number with decimal notation (e.g., 392.55)
 - **REGEX:** [0-9]+[0-9\.\,][0-9]+

The REGEX rule elements are grouped using square brackets. Meta characters such as commas, question marks, and dollar signs must be preceded by a backslash to indicate that they have a literal meaning. For example, [\?] matches a question mark in text. You cannot refer to concepts in a REGEX expression. You can match one or more consecutive digits without decimals, such as matching a number with decimal notation (for example, 392.55).

This rule matches any digit 0 to 9, followed by either a digit, a comma, or a period, and then followed by any number of digits.

REGEX Matching, Guidelines, and Examples

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Regular Expressions: A Brief History

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Using the REGEX Rule

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: REGEX Rule



Copyright © SAS Institute Inc. All rights reserved.

Question: REGEX Rule

Which REGEX rule has the correct syntax to extract a part number that starts with XYZ- and any five-digit numbers (for example, XYZ-12345)?

- a. REGEX:[XYZ][\-][1234567890]
- b. REGEX:XYZ[\-][1234567890]+
- c. REGEX:[XYZ][\-][\d\d\d\d\d]
- d. REGEX:XYZ\-\d\d\d\d\d

Answer: REGEX Rule

Which REGEX rule has the correct syntax to extract a part number that starts with XYZ- and any five-digit numbers (for example, XYZ-12345)?

- a. REGEX:[XYZ][\-][1234567890]
- b. REGEX:XYZ[\-][1234567890]+
- c. REGEX:[XYZ][\-][\d\d\d\d\d]
- ☒ d. REGEX:XYZ\-\d\d\d\d\d

Answer: d

The square brackets around a character string such as XYZ will only match one of the characters in the string. Option A will match X-1, but not XYZ-12345. A similar argument applies to option C. Option B will match XYZ-1234567890123456789, in other words, there is no limit on the number of digits. Note that the square brackets around a single character has no effect. Thus, [\-] will work, but is not preferred.

Resource

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



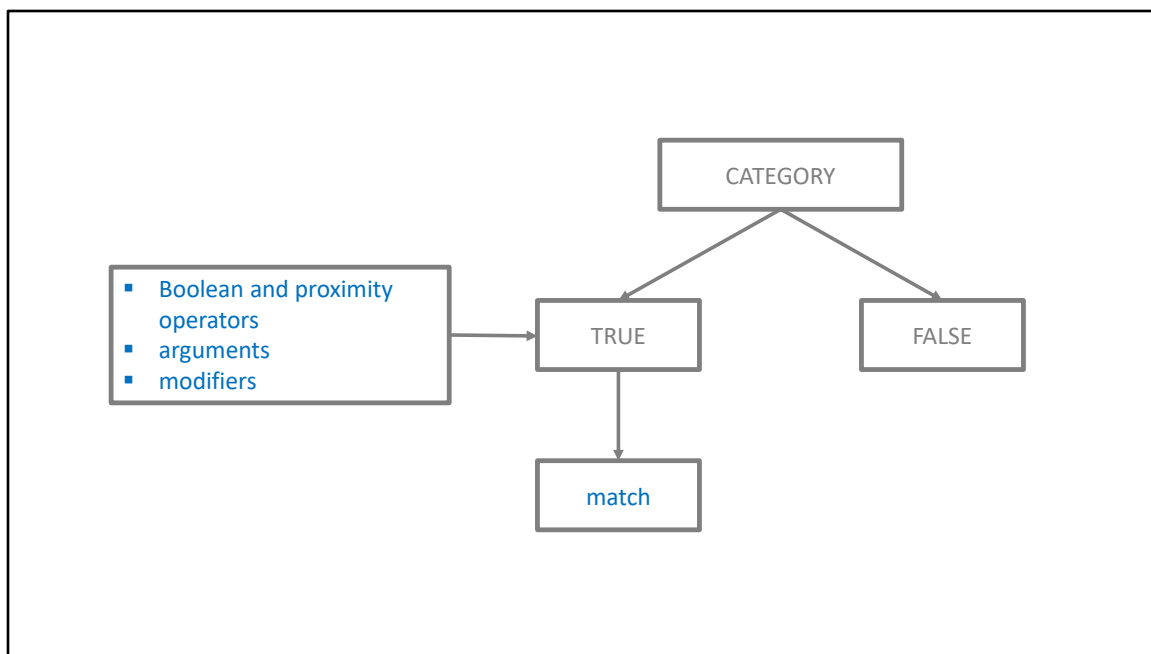
4.3 SAS Visual Text Analytics Category Rules



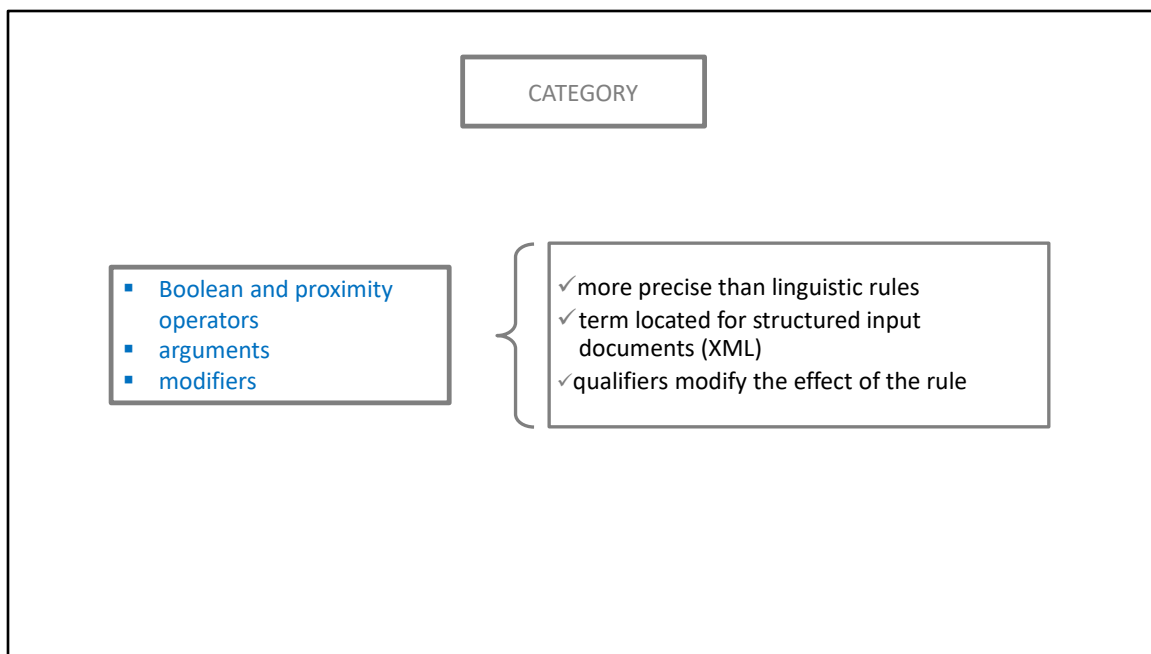
Introduction to CATEGORY Rules



Copyright © SAS Institute Inc. All rights reserved.



CATEGORY rules resolve to True or False, and True results in a match. CATEGORY rules use Boolean and proximity operators, arguments, and modifiers to define the conditions that are necessary for category matches. These rules are simpler to write than LITI rules and are recommended when there is no need to extract specific information from the data.



You can use Boolean operators to write more precise and unambiguous rules. Boolean rules can also be used when a term is located for structured input documents such as XML in order to return a match. As with linguistic rules, you can add qualifiers to modify the effect of the rule.

CATEGORY

(**OPERATOR**, *argument1*, *argument2*,...)

(AND, "AMAZON", (OR, "online retailer", "e-commerce"))

For a CATEGORY rule, use the syntax **OPERATOR**, **argument1**, **argument2**, where arguments can be terms, strings, or nested rules. Boolean and proximity operators and their arguments are enclosed in parentheses and separated with commas. The arguments are included in quotation marks. Rules can be nested.

In the example, the rule is true when the term *Amazon* appears with either the terms *online retailer* or *e-commerce*.

CATEGORY

Symbol	Position	Result
@	Suffix	Stems the word that precedes the @ to include <i>all forms</i> of the word.
@N	Suffix	Stems the word that precedes the @N to include <i>noun forms</i> of the word.
_C	Suffix	Makes the rule case-sensitive.
@V	Suffix	Stems the word that precedes the @V to include <i>verb forms</i> of the word.
*		Wildcard matching – Matches any characters that occur at the beginning or end of the word. For example, the argument "travel*" returns the matches <i>travels</i>, <i>traveled</i>, <i>traveler</i>, and <i>traveling</i>. The argument "*room" matches <i>bedroom</i>, <i>cloakroom</i>, <i>ballroom</i>, <i>room</i>, and so on.

You can add special symbols to terms that are specified in Boolean rules to expand word forms, perform case-sensitive matching, and specify a wildcard. A wildcard matches any characters that occur at the beginning or end of the word. For example, the argument "travel*" returns the matches *travels*, *traveled*, *traveler*, *traveling*, and so on. The argument "*room" matches *bedroom*, *cloakroom*, *ballroom*, *room*, and so on.

CATEGORY Boolean Rule Syntax and Examples

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Using the CATEGORY Rule

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: CATEGORY Rule



Copyright © SAS Institute Inc. All rights reserved.

Question: CATEGORY Rule

Which sub-category Boolean rule has the incorrect syntax?

- a. (AND, (NOT, (OR, "Enterprise Guide", "DDE", "Drug")), (OR, "Business Intelligence", "Enterprise Miner"))
- b. (SENT, (NOT, (OR, "Text Miner", "REG", "GLM")), (OR, "Business Intelligence", "Enterprise Miner"))
- c. (SENT, ["WAR_CONCEPT"], "europe")
- d. (ORD,_tmac:"@NAME/FIRST",_tmac:"@NAME/LAST")

Answer: CATEGORY Rule

Which sub-category Boolean rule has the incorrect syntax?

- a. (AND, (NOT, (OR, "Enterprise Guide", "DDE", "Drug")), (OR, "Business Intelligence", "Enterprise Miner"))
- b. (SENT, (NOT, (OR, "Text Miner", "REG", "GLM")), (OR, "Business Intelligence", "Enterprise Miner"))
- ☒ c. (SENT, ["WAR_CONCEPT"], "europe")
- d. (ORD,_tmac:"@NAME/FIRST",_tmac:"@NAME/LAST")

Answer: c

Square brackets [] should be used to reference concepts instead of curly braces {}.

Practice: Creating Appropriate Concept or Category Rules

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.

